Week 1 Cybersecurity Testing Report

Student Name: Sadia Tahir
Module: Cybersecurity Internship – Week 1
Date: 5th March 2026
Target Applications:

- Manual Testing: https://demo.testfire.net
- Automated Testing: http://testphp.vulnweb.com (OWASP ZAP)

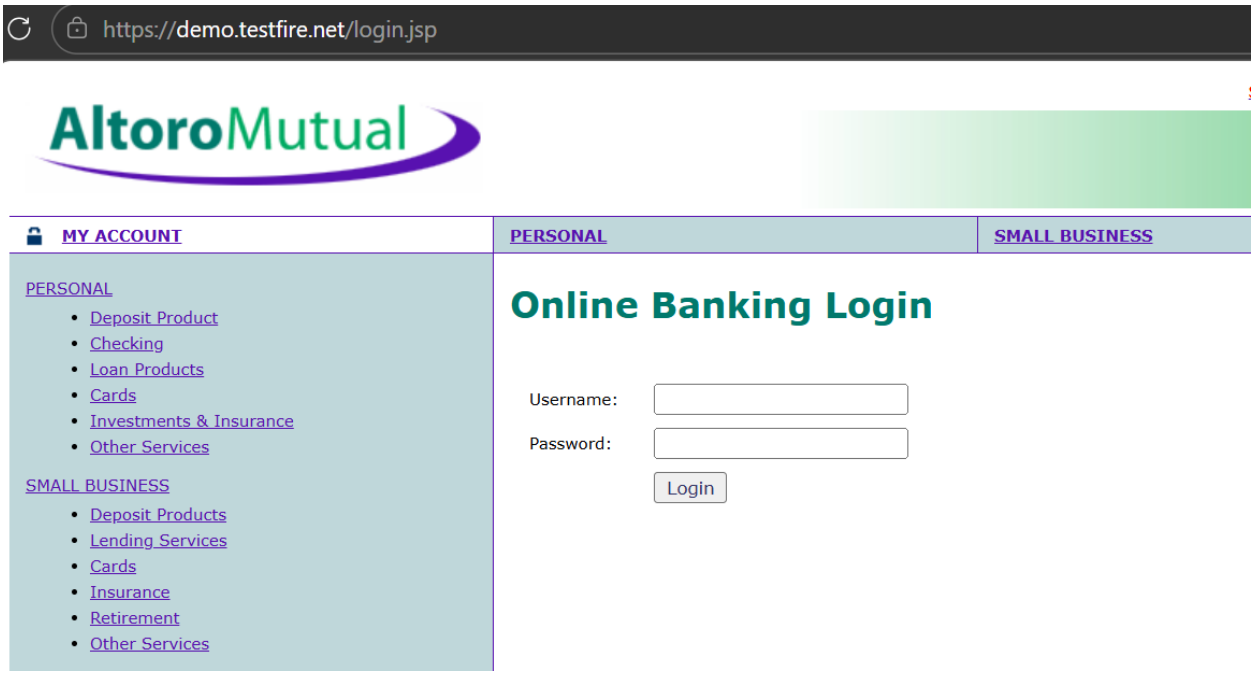
1. Objective / Focus Areas

- Explore the web applications to understand functionality.
- Identify basic vulnerabilities using manual testing and OWASP ZAP.
- Focus Areas:
  - Cross-Site Scripting (XSS)
  - SQL Injection
  - Weak password storage
  - Security misconfigurations

2. Manual Testing (demo.testfire.net)

Steps Taken:

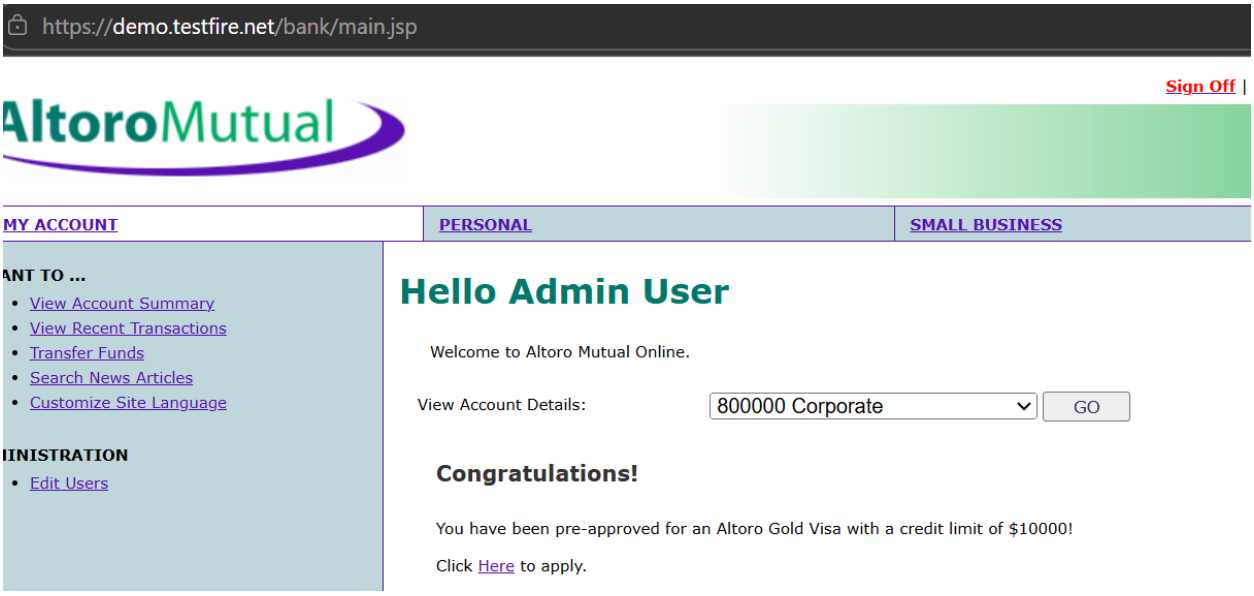
1. Open the Login page
   https://demo.testfire.net/login.jsp



2.Check normal login with sample credentials.
3.Test Direct Page Access:

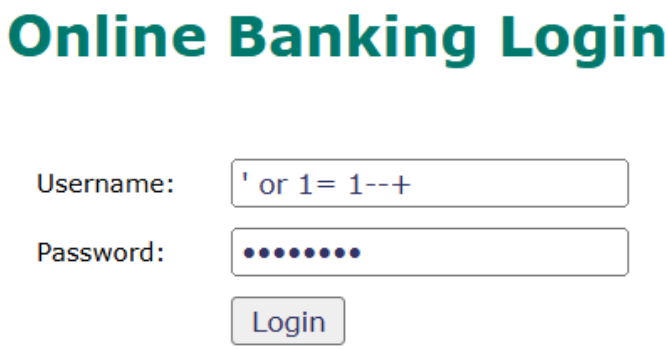
https://demo.testfire.net/bank/main.jsp

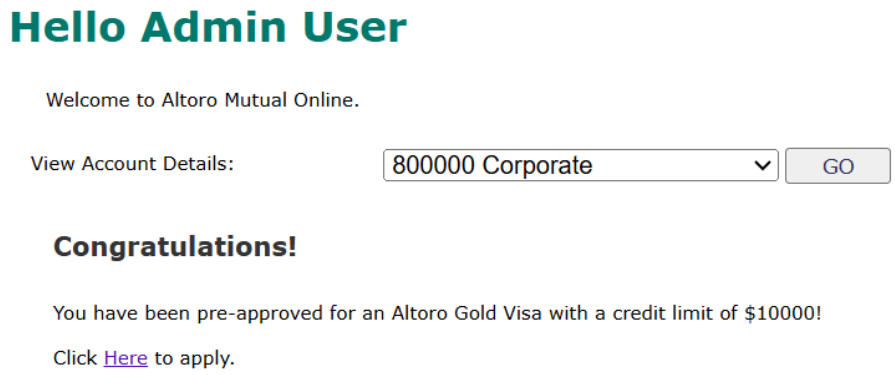
Observation: Admin page opens without login.



4.SQL Injection Test:

- **Username:** `' or 1=1--+`
- **Password:** `1234567`
- **Observation:** Authentication bypass achieved; admin account accessed.





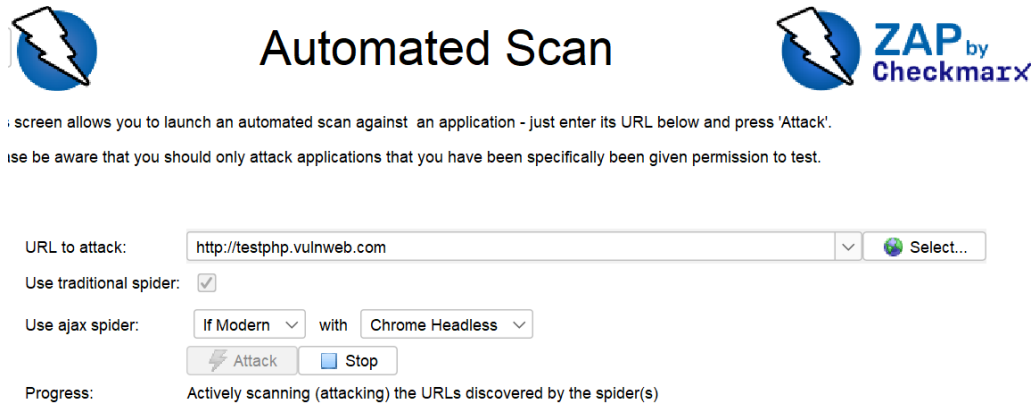
5.Other checks:

- XSS attempts (no immediate XSS found)
- Password storage not directly visible

Vulnerabilities Found

#	Vulnerability	Description	Example/Payloads	Impact
1	Broken Access Control	Direct URL access to sensitive pages without login	https://demo.testfire.net/bank/main.jsp	User can access admin/dashboard
2	SQL Injection	Authentication bypass using SQL query manipulation	`' or 1=1--+` in username	Admin account access

2. Automated Testing (OWASP ZAP – testphp.vulnweb.com)



Summary of Findings:

Risk Level	Count	Example
Medium	3	CSP header not set, Missing Anti-clickjacking header, Absence of Anti-CSRF tokens
Low	6	Application error disclosure, Server leaks info, X-Content-Type-Options missing, ZAP outdated
Informational	2	Charset mismatch, User Agent Fuzzer

Screenshots :

Content Security Policy (CSP) Header Not Set

URL: http://testphp.vulnweb.com
Risk: Medium
Confidence: High
Parameter:
Attack:
Evidence:
CWE ID: 693
WASC ID: 15
Source: Passive (10038 - Content Security Policy (CSP) Header Not Set)
Alert Reference: 10038-1
Input Vector:

Description:

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a

Application Error Disclosure

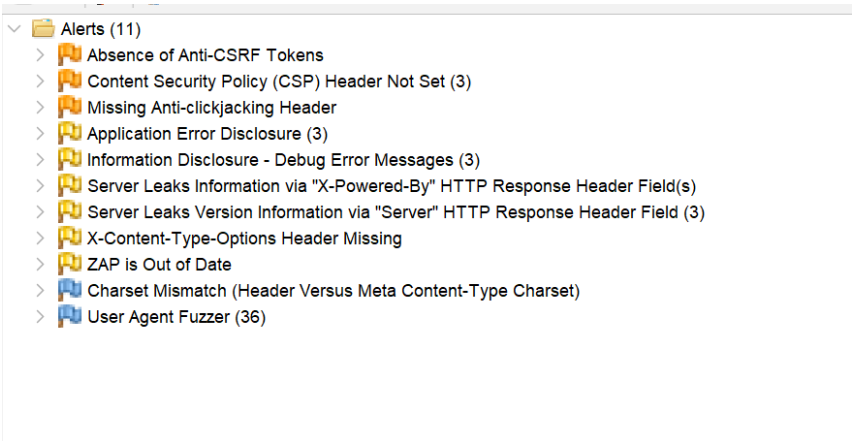
URL: http://testphp.vulnweb.com/robots.txt
Risk: Low
Confidence: Medium
Parameter:
Attack:
Evidence: HTTP/1.1 500 Internal Server Error
CWE ID: 550
WASC ID: 13
Source: Passive (90022 - Application Error Disclosure)
Input Vector:

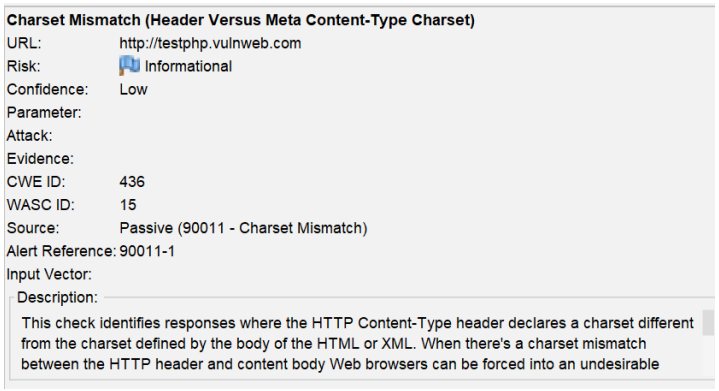
Description:

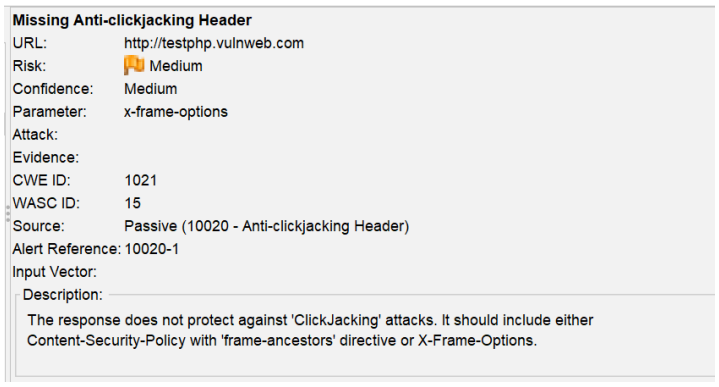
This page contains an error/warning message that may disclose sensitive information like the location of the file that produced the unhandled exception. This information can be used to launch further attacks against the web application. The alert could be a false positive if the error message

Key Alerts:

1. Content Security Policy (CSP) Header Not Set – Risk: Medium
2. Missing Anti-clickjacking Header – Risk: Medium
3. Absence of Anti-CSRF Tokens – Risk: Medium
4. Application Error Disclosure – Risk: Low
5. Server Leaks Information via Headers – Risk: Low
6. Charset Mismatch / User Agent Fuzzer – Informational







4. Areas of Improvement

1. Implement proper access control on all sensitive pages.
2. Sanitize input fields to prevent SQL Injection.

3. Set security headers: CSP, X-Frame-Options, X-Content-Type-Options.
4. Use hashed & salted passwords.
5. Fix debug info / error disclosure issues.
6. Regularly update tools (e.g., ZAP) and patch web server vulnerabilities.

Key Findings

- Manual SQL Injection test succeeded: `' OR 1=1 --+` allowed access to admin account.
- OWASP ZAP scan found Medium risk issues: missing Content Security Policy (CSP), missing Anti-CSRF tokens, missing Anti-clickjacking header.
- Low risk issues detected: server version leakage, debug messages, X-Content-Type-Options missing.
- Informational issues: character set mismatch, user agent fuzzer events.
- Web application shows weak security configurations and needs basic hardening.

Conclusion

The web application demonstrates critical learning points for interns: manual testing identified SQL injection vulnerability while automated ZAP scan highlighted common security misconfigurations. Immediate improvements are recommended in input validation, HTTP headers, and CSRF protections. Implementing these measures will strengthen the security posture and reduce potential exploitation risks.